

Mazzaroth: A High-Throughput DAG Consensus with State Root

abstract

比特币的 Nakamoto Consensus 通过单链区块链实现去中心化账本，假设网络最大延迟 Δ ，限制出块速度，导致低吞吐量。PHANTOM GHOSTDAG (PG) 通过 blockDAG 提升吞吐量，但其概率性确认限制了智能合约应用。为此，Mazzaroth 提出基于 PoW 的 blockDAG 共识，通过线性排序算法计算状态根，实现状态最终性，支持智能合约。其动态难度调整无需 Δ 假设，自适应网络和算力波动，通过 head 链确保状态一致性，最大化吞吐量。模拟测试验证了 Mazzaroth 的高效共识性能。本文介绍 Mazzaroth 的排序算法、难度调整机制及性能评估

1 INTRODUCTION

比特币的 Nakamoto Consensus 通过单链区块链实现去中心化账本，假设网络直径延迟上界 Δ （任意两节点间最大传播延迟），并设定预期出块时间 λ 远大于 Δ 以确保区块传播。然而，这限制了出块速度，导致交易吞吐量低，且难以适应网络延迟波动。减小 λ 损害去中心化，增大 λ 则降低吞吐量，形成两难困境。

PHANTOM GHOSTDAG (PG) 采用 blockDAG 结构提升吞吐量，其算法通过交易排序解决共识问题，但确认具有概率性，状态计算复杂，限制了智能合约应用，且缺乏动态难度调整。

Mazzaroth 提出基于 PoW 的 blockDAG 共识，通过排序算法实现快速状态最终性，支持智能合约。其动态难度调整无需 Δ 假设，自适应网络波动，利用 head 链形成状态根，将吞吐量推向网络极限。本文介绍首先介绍 Mazzaroth 的排序算法，然后通过一些性能测试指标，引导出 Mazzaroth 的难度调整机制。

2 Mazzaroth 协议：排序算法

2.1 Preliminaries

本文使用了以下术语，一个有向无环图 DAG 表示为 $G=(C,E)$ ，其中 C 表示区块， E 表示区块之间的 hash 引用，我们经常写 $B \in G$ 而不是 $B \in C$ 。

$\text{parent}(B,G) \subset C$ 表示 B 的双亲节点的集合

$\text{past}(B,G) \subset C$ 表示从 B 可达的区块子集（不包括 B ）。

$\text{size}(B,G) \subset C$ 表示 $\text{past}(B,G)$ 中区块的数量+1（包含 B 本身）。

$\text{anticone}(B,G) \subset C$ 表示不在 $\text{past}(B,G)$ 中的节点

$\text{tips}(G) \subset C$ 表示叶子结点，即入度为 0 的区块。

值得注意的是，当区块 B 被挖掘出来后， $\text{past}(B,G)$ 和 $\text{size}(B,G)$ 就被确定了，无论后续诚实/恶意矿工做什么都没办法改变。

Figure 1:

$\text{parent}(F,G)=\{E,D\}$

$\text{past}(F,G)=\{\text{Genesis}, A, B, C, D, E, F\}$

$\text{size}(F,G)=7$

$\text{anticone}(F,G)=\{H\}$

$\text{tips}(G)=\{H,F\}$

2.2 Mazzaroth排序协议

Mazzaroth和PHANTOM协议一样，即使两个区块包含冲突的交易，这两个区块都会被纳入blockDAG并被所有矿工引用。

大致上来说，Mazzaroth分为一下几个步骤

- (1) 找到 $\text{tips}(G)$ 中 $\text{size}(B)$ 最大的一个区块，然后定义为Head。
- (2) $\text{past}(\text{Head})$ ，进行递归计算排序
- (3) 对于不在 $\text{past}(\text{Head})$ 中的区块，尊重拓扑排序，并根据区块size排序。然后把这些区块按顺序放在 $\text{past}(\text{Head})$ 的排序后
- (4) 输出交易的排序

2.2.1 排序协议的形式化算法

上面描述的程序在下面算法中形式化。算法的输入是一个BlockDAG，输出是an ordered list containing all blocks in G。算法的最开始，即DAG仅由创世区块组成（第2-3行），接下来对DAG的每个叶区块(tips)的过去进行查找，找到 $\text{size}(B,G)$ 最大的，并继承其排序。随后将此节点也放入排序(4-7行)。随后使用拓扑排序，将连接状态不好的节点放在后面(8-11行)，随后将当前节点加入排序，最后合并排序（12行）

Algorithm 1 Ordering the DAG

Input: G - a block DAG

Output: an ordered list containing all blocks in G

function OrderDAG(N,G)

 if $G == \{\text{genesis}\}$ then

 return [genesis]

 for $B \in \text{parent}(N,G)$ do

 [SetSize] $\leftarrow \text{size}(B,G)$

$B_{\max} \leftarrow \text{argmax}(\text{SetSize}B: B \in \text{tips}(G))$

 OrderedList $\leftarrow \text{OrderDAG}(B_{\max}, \text{past}(B_{\max}))$

 EndList $\leftarrow []$

 for $B \in \text{anticone}(B_{\max}, G)$ 使用逆拓扑序，优先选择 $\text{size}(B,G)$ 更小的节点

 add B to the front of EndList

 add EndList to the end of OrderedList

 add N to OrderedList

 return OrderedList

3 指标

3.1 交易

我们定义一个交易主要以下几个部分组成（省略与排序算法无关的信息）

输入：多个账户余额

输出：多个账户余额（包含所有输入账户，即使此时账户余额为0）

签名：每个余额减少账户的签名

约束：输入总余额等于输出总余额。

交易有效性取决于输入账户余额是否与交易声明一致。账户可设置最小接收余额以防止 DDoS 攻击。基于排序算法生成的交易序列，Mazzaroth 依次遍历交易，计算全局状态根（state root）。Mazzaroth 虚拟机支持状态回溯：从前向后推导时，若交易输入余额有效，则接受该交易，否则忽略；从后向前回溯时，若交易输出余额有效，则接受，否则忽略。此机制使得共识算法可以通过当前区块的状态根，推导出之前的区块的状态根。使得状态根在 BlockDAG 中不同的区块之间相互转换成为可能。

3.2 参数和测试

以下指标用于评估 Mazzaroth 共识性能：

$\text{head}(B, G)$: $\text{parent}(B, G)$ 中 size 最大的那个区块

$\text{link}(B, G)$: 为 $\text{head}(B, G)$ 一直向上回溯的区块的集合，即 $\{\text{head}(B, G), \text{head}(\text{head}(B, G), G), \text{head}(\text{head}(\text{head}(B, G), G), G), \dots \text{Genesis}\}$

$\text{agree}(\{B, P\}, G)$ 为 $\text{link}(B, G)$ 和 $\text{link}(P, G)$ 两个集合交集中，size 最大的区块

$\text{distance}(B, G)$ 为 B 的 size 与 B 的 parent 的 agree 的 size 的差值。 $\text{size}(B, G) - \text{size}(\text{agree}(\text{parent}(B, G), G), G)$ 。

$\text{state}(B, G)$ 为 $\text{past}(B, G)$ 以及区块 B 所构成的集合，集合计算出排序，计算出的 mazzaroth 虚拟机的状态根

在排序算法中， $\text{state}(B, G)$ 继承自 $\text{state}(\text{head}(B, G), G)$ 。且 $\text{parent}(B, G)$ 中所有节点的 state 都可以相互推导。当 $\text{distance}(B, G)$ 较小时，网络达成共识；当 $\text{distance}(B, G)$ 较大（如 $\text{agree}(\text{tips}(G), G) = \text{genesis}$ ），表明共识失效，可能发生网络分裂。

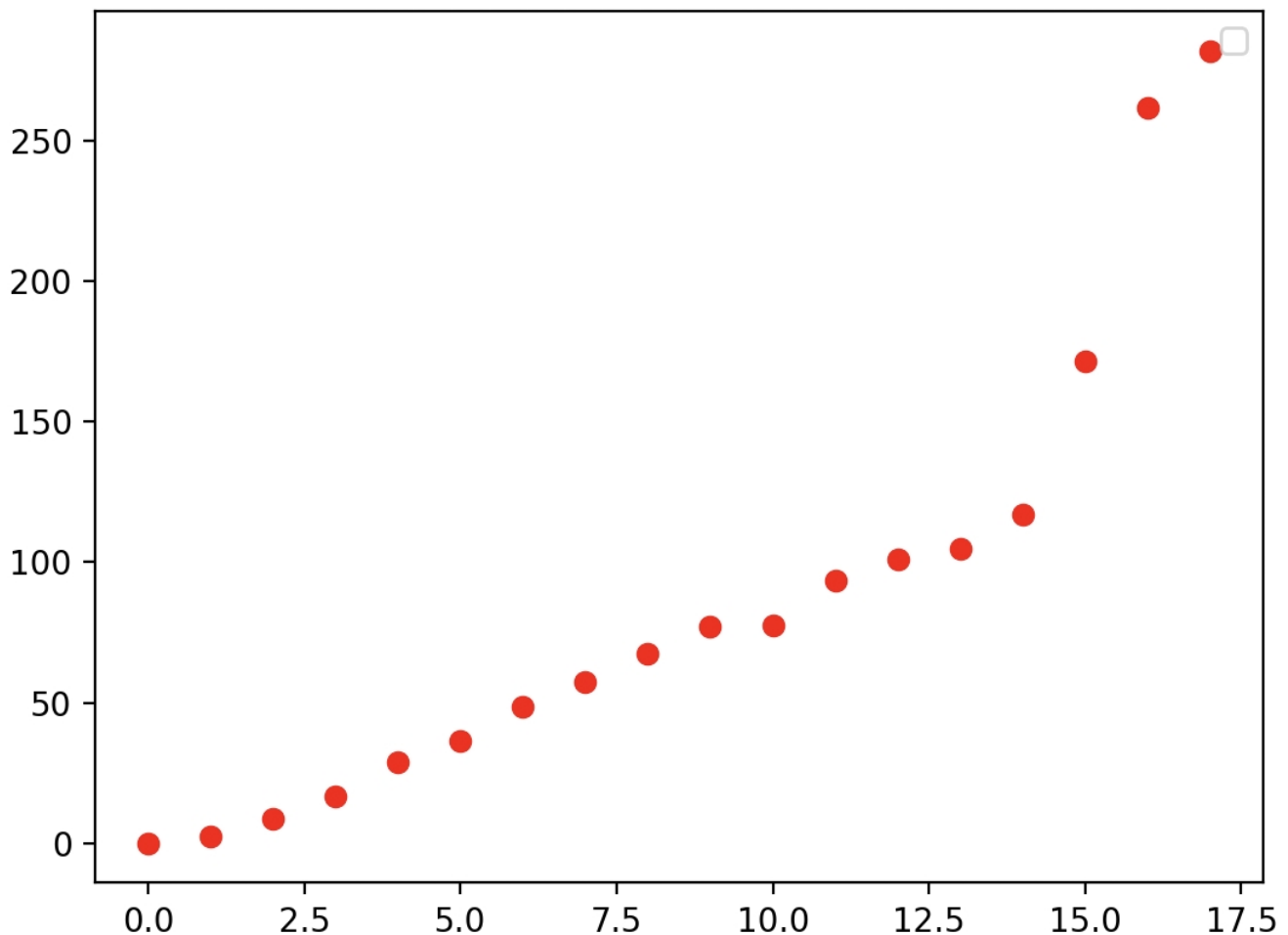
定义网络在动态网络直径延迟 D' （区别于固定上界 Δ ）内挖掘的区块数为 Block Per Delay（BPD）， $\text{distance}(B, G)$ 简称 DIS。BPD 与 DIS 正相关

模拟测试使用 1000 个矿工（900 个在 D' 内同步，100 个延迟同步），共挖掘 10000 个区块，记录不同 BPD 下 DIS 的变化（代码见 <https://github.com/mazzarothnet/mazzaroth>）。图 3.1 显示 DIS 随 BPD 增加而增长的趋势。图 3.2 显示 $\text{agree}(\text{tips}(G), G)$ 的 size 与 $\text{tips}(G)$ 中最大 size 的差值，发现当 $\text{BPD} > 14$ 时，agree 常为 genesis，表明共识分裂。

“Pasted image 20250503215216.png” could not be found.

图 3.1: DIS 随 BPD 的变化趋势。

描述：展示 DIS 在不同 BPD 值下的增长，反映网络共识稳定性。



Difference Between size of $\text{agree}(\text{tips}(G), G)$ and Maximum size in $\text{tips}(G)$

图 3.2: $\text{agree}(\text{tips}(G), G)$ size 与 $\text{tips}(G)$ 最大 size 的差值。

描述：当 $\text{BPD} > 14$ 时， agree 退化为 genesis ，表明网络分裂。

4 Mazzaroth协议-难度调整算法

难度调整算法与排序算法同等重要，共同构成 Mazzaroth 的核心机制。本节介绍 Mazzaroth 如何通过动态难度调整优化 blockDAG 结构，支持高效状态根计算和最大化吞吐量。

4.1 设计理念

比特币的 Nakamoto Consensus 通过难度调整控制区块生成时间（约 10 分钟），假设网络最大延迟 Δ ，确保区块传播并形成单链结构，链上状态根代表共识。然而，这一机制限制了吞吐量。Mazzaroth 基于 blockDAG 结构，目标是使 head 链形成稳定序列，最长 head 链的状态根表示共识，等效于维持 $\text{distance}(B, G)$ (DIS) 在有限范围内。测试表明（见 3.2），DIS 与 Block Per Delay (BPD，即动态网络延迟 D' 内的区块数) 正相关。Mazzaroth 通过观察 DIS 动态调整 PoW 难度，无需记录时间戳，优化 blockDAG 结构。

4.2 算法描述

Mazzaroth 的难度调整基于拟合函数 $f(\text{DIS}) = \text{BPD}$ ，反映 DIS 和 BPD 的对应关系。算法设定预期 BPD (PBPD)，表示在不同网络延迟和算力下的目标区块生成速率。PoW 难度通过 $\text{SHA256} < \text{target}$ 定义，调整公式如下：

公式 4.1：

$$\text{target} = \text{target}' * \text{PBPD} / f(\text{DIS}')$$

其中， target' 为旧难度， DIS' 为当前观察的 DIS，PBPD 为预期 BPD。

Mazzaroth 使用两个难度目标：

target1：控制长期波动，由全网算力主导，每 $2016 \times 10 \times 60$ 个区块调整一次，基于区块难度和 DIS 的平均值。

target2：控制短期波动，由网络延迟主导，每个区块根据其 parent 的 DIS 和难度平均值调整。区块的 SHA256 需同时满足 target1 和 target2 才被接受。

4.3 应用场景

PBPD 反映共识速度与吞吐量的权衡。小 PBPD 降低 DIS，加快交易确认，适合低延迟场景；大 PBPD 增加 DIS，提升吞吐量，适合高负载应用。实际部署中，可根据需求调整 PBPD，优化 Mazzaroth 的性能。

5 总结

Mazzaroth 提出了一种基于 blockDAG 的共识协议，针对传统 blockDAG 难以高效计算状态根的挑战，设计了新颖的排序算法和动态难度调整机制。排序算法通过线性排序生成交易序列，支持状态根的快速计算；难度调整算法通过控制 Block Per Delay (BPD)，优化 blockDAG 结构，维持有限的 distance (DIS)。

模拟测试验证了 Mazzaroth 的高效性能，显示其在高 BPD 下最大化吞吐量，同时确保交易快速确认和状态最终性（见 3.2）。在诚实节点控制多数算力的假设下，Mazzaroth 实现高吞吐量共识，支持智能合约应用。